CHAPTER 5.

What is Unicode little endian

ACTIVE DIRECTORY FICAM PROTOCOLS AND THE MECHANICS OF TRUST: LM, NTLM, AND KERBEROS

ABSTRACT

Authentication is the moment an enterprise identity system makes its most consequential decision: whether a presented credential corresponds to a real, legitimate principal, and whether that identity should be trusted to execute operations within the security boundary. Active Directory (AD) does not make this specific decision once at interactive logon and walk away; it evaluates trust continuously across network boundaries. It does this through two fundamentally distinct authentication protocol families that handle nearly all work in a Windows domain: NTLM and Kerberos. These protocol families coexist in virtually every enterprise environment I've ever assessed or penetrated, yet they share different design philosophies, manifest unique cryptographic weaknesses, and demand entirely different defensive engineering strategies.

Chapter 5 deconstructs both protocol families at the byte and structural level. We begin with LAN Manager (LM) and New Technology LAN Manager (NTLM), tracing their hash construction, challenge-response state machines, and the critical operational boundaries between the NT hash (which enables Pass-the-Hash (PtH) and the NetNTLM response (which supports offline brute-forcing, cracking, and relaying). We then dissect the Kerberos v5 architecture – tracing the complete five-message exchange from `AS-REQ` to `AP-REP`, the structural layout and signing verification of the Privilege Attribute Certificate (PAC), Service Principal Name (SPN) directory mapping, and the operational nuances of Active Directory delegation models. Finally, we cross-examine the attack surfaces of both protocols and establish the deterministic event log telemetry required for modern robust identity continuous monitoring.

KEYWORDS

NTLM; Kerberos; LAN Manager; Authentication; Privilege Attribute Certificate (PAC); Service Principal Name (SPN); Delegation; Ticket Granting Ticket (TGT); Encryption types

KEY TERMS

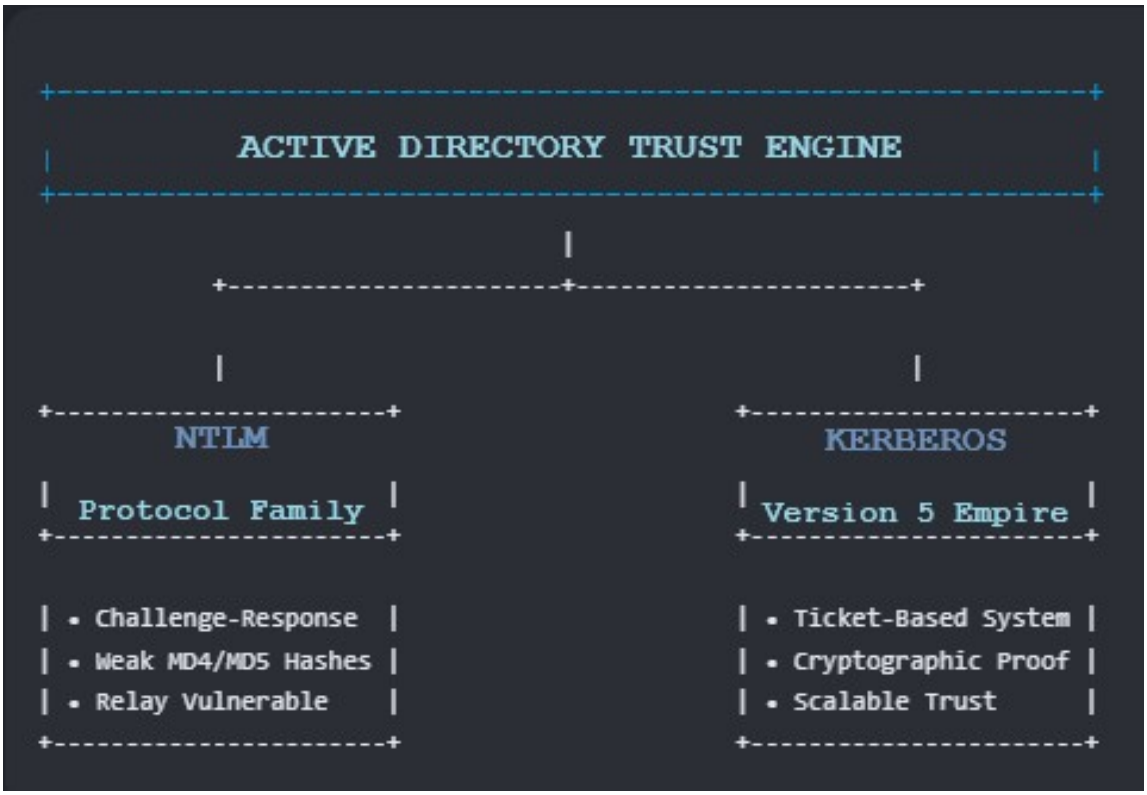
- **LAN Manager (LM) Hash:** A legacy cryptographic hash of a Windows password created using a Data Encryption Standard (DES)-based algorithm that converts the

password to uppercase, pads it to 14 characters, splits it into two 7-character halves, and encrypts each half separately – making it trivially crackable.

- **NT Hash (`NTHash`):** A Message Digest 4 (MD4) hash of the Unicode little-endian encoded representation of a password. Serves as the key material for both NTLM authentication and Kerberos Rivest-Cipher 4 (RC4) encryption. This is the hash that can be passed directly in Pass-the-Hash (PtH) attacks.
- **NetNTLMv2:** The challenge-response value computed during NTLM authentication – specifically the HMAC-MD5 of the server challenge combined with a client challenge, keyed with the NT hash. NetNTLMv2 cannot be passed; it can only be cracked offline or relayed to another service in real-time.
- **Key Distribution Center (KDC):** The Kerberos service runs on a domain controller, consisting of two logical components: the Authentication Service (AS) that issues Ticket Granting Tickets (TGTs), and the Ticket Granting Service (TGS) that issues service tickets.
- **Ticket Granting Ticket (TGT):** A Kerberos ticket issued by the Authentication Service (AS), encrypted with the `krbtgt` account's long-term key. The client presents it to the TGS to obtain service tickets without re-entering credentials.
- **Service Ticket (ST):** A Kerberos ticket issued by the Ticket Granting Service (TGS) for a specific domain or network-based service, encrypted with that service account's long-term key. The client presents it to the target service to prove identity.
- **Privilege Attribute Certificate (PAC):** A Microsoft extension embedded in Kerberos tickets that carries Windows-specific authorization data and information: the user's Security Identifier (SID), group memberships, logon information, and credential metadata. Signed by both the KDC and the service to detect tampering.
- **Service Principal Name (SPN):** A unique identifier in the format `serviceClass/host:port` that maps a service to the account running it, enabling the KDC to identify which account's key to use when issuing a service ticket.
- **`krbtgt`:** The Key Distribution Center service account whose password-derived key encrypts all issued Ticket Granting Tickets in the domain. Compromise of this key enables Golden Ticket forgery, and many consider it to be the "crown jewel" of Active Directory while others argue and consider the true "crown jewels" of Active Directory to be the domain's lifeblood: the "dit," or "dit file." (`ntds.dit`).
- **Unconstrained Delegation:** A Kerberos configuration that permits a service to request Kerberos tickets for any other service on behalf of an authenticating user – one of the most dangerous misconfigurations in an Active Directory environment.

5.1 WHY AUTHENTICATION PROTOCOLS ARE ATTACK SURFACE

Active Directory is more than a directory services database; it is the definitive trust engine of the entire enterprise. Every time a principal accesses a network resource – whether mapping a Server Message Block (SMB) file share, authenticating to a web application, or establishing a Remote Procedure Call (RPC) session – the transaction relies on a defined sequence of cryptographic operations and message exchanges to establish mutual identity validation.



Defenders frequently fail to prevent identity-based attacks – such as Kerberoasting, AS-REP Roasting, Pass-the-Hash (PtH), or NTLM Relay – not because they do not recognize the names of the techniques, but because they do not fully understand the underlying protocol mechanics that support the entire operational architecture. Without a low-level grasp, or basic comprehension of which bits are sent “over the wire,” how keys are derived, and which components are sealed by which forest-level authoritative identities, and, unless you can foresee future events, it is practically and realistically impossible to predict what type of impact an attack may impart on a victim environment, or the post-compromise attack evidence and artifacts that may be left behind, which controls will successfully mitigate a threat, and which configurations merely create a false sense and an illusion of absolute security. This will never exist.

In a Windows Active Directory environment, virtually all heavy network lifting is shouldered by two protocol families: NTLM and Kerberos. Make no mistake, they are not interchangeable. They suffer from completely different vulnerabilities, fail in entirely unique ways, and boast

vastly different attack surfaces and patch cadences. The mechanics of how they authorize human users versus non-person entities (NPEs) do not align.

Yet, despite these glaring differences, both protocols coexist in almost every environment I have ever assessed. They are routinely implemented without compelling justification or clear boundaries dictating when one takes over from the other. Attackers know this better than anyone, and they actively aim to exploit the friction and overlap between the two.

For a defender, understanding only one of these protocols means operating with just half the picture.

Chapter 5 does not rush past the mechanics. The reason most practitioners fail to detect or prevent authentication-based attacks is not ignorance of the attack names – Kerberoasting, Pass-the-Hash (PtH), NTLM relay are all well-known labels. The reason is not understanding the underlying mechanics well enough to recognize what the attack is preparing to do to the protocols, and therefore what signals it leaves behind, and which controls are known to stop it versus which ones only appear.

That understanding starts here.

5.2 LAN MANAGER: THE PROTOCOL THAT REFUSES TO RETIRE

Created in the late 1980s for OS/2 and early DOS environments, LAN Manager (LM) was engineered for computational efficiency on 16-bit hardware rather than cryptographic resistance. It predates modern threat modeling and the assumption of untrusted local network segments. While Microsoft disabled LM hash storage by default beginning with Windows Vista and Server 2008, legacy technical debt, misconfigured Group Policies, and un-audited enterprise acquisitions frequently cause LM authentication negotiation to persist inside modern subnets. As such, LM authentication was disabled by default around the same time. This means that in a modern, properly configured and hardened as feasibly as possible, LM should be completely non-existent in a modern domain environment; however, that is not always the case, and, more times, than not – it remains.

5.2.1 HOW THE LM HASH IS CONSTRUCTED

The structural design of the LM hash represents a textbook case of architectural fragility. The derivation process occurs via three strict algorithmic steps:

5.2.1.1 a. CASE CONVERSION

The cleartext password is converted to uppercase. `P@ssword123` becomes `P@SSWORD123`. This immediately collapses the cryptographic keyspace by removing the entire lowercase character set.

5.2.1.2 b. PADDING AND TRUNCATION

The uppercase string is padded with null bytes (`\x00`) to exactly 14 characters. If a password exceeds 14 characters, it is truncated to the first 14 characters, completely discarding the remainder of the entropy.

5.2.1.3 c. SPLIT DES ENCRYPTION

The 14-byte padded string is split into two independent 7-byte (56-bit) chunks. Each 7-byte chunk is utilized as a Data Encryption Standard (DES) key to encrypt a fixed, publicly known plaintext constant string: `KGS!@#$%`. Each DES operation outputs an 8-byte ciphertext block. These two 8-byte blocks are concatenated to yield the final 16-byte LM hash.

5.2.1.4 d. CRYPTOGRAPHIC IMPLICATIONS

This is considered catastrophic, in my book. The two halves are encrypted completely independently. An attacker does not need to crack the whole password at once – they crack each 7-character half separately, reducing a 14-character search for two parallel 7-character searches. If a password is 7 characters, or shorter, the second 7-byte chunk consists entirely of null bytes. This means the second half of the LM hash will always evaluate to the static cryptographic constant value: `AAD3B435B51404EEAAD3B435B51404EE`. When an analyst or attacker spots this exact value in a credential dump, it serves as an immediate structural disclosure that the plaintext password is indeed 7 characters or fewer, rendering the remaining half trivial to resolve via a standard Graphics Processing Unit (GPU) in mere milliseconds.

Legacy applications that authenticate against older servers, embedded systems that were never updated, environments that restored a Group Policy setting to “fix” a compatibility problem years ago, and hybrid environments that absorbed acquisitions without auditing their baselines – all of these regularly contain systems that still negotiate LM. A single host in a subnet that responds to LM challenges can funnel authentication attempts through the weakest possible path.

TL;DR: The LM hash is fundamentally broken by design. Its construction guarantees rapid offline cracking and recovery. If an LM hash exists anywhere in an environment, the underlying password is most definitely recoverable.

5.3 NTLM: THE CHALLENGE-RESPONSE PROTOCOL

The New Technology LAN Manager (NTLM) suite was introduced with Windows NT 3.1 to rectify the structural vulnerabilities of LM. NTLM encompasses the raw NT hash and two wire-protocol implementations: NTLMv1 and NTLMv2 - that share a common hash construction but differ significantly in how they compute the challenge-response value sent across the network. Understanding both versions is necessary because NTLMv1 remains in some environments despite being functionally obsolete, and because the distinction between the two determines which attacks are available and which defensive controls actually matter. The subsections below begin with the NT hash itself — the raw credential material both versions derive from — then

trace the authentication mechanics of each version in sequence, and close with the operational distinction between the NT hash and the Net-NTLMv2 response that practitioners most commonly confuse.

5.3.a NT HASH CONSTRUCTION AND CRYPTOGRAPHIC DEFICIENCIES

The NT hash (commonly termed the NTHash) serves as the core authentication secret stored in the Active Directory database (`ntds.dit`) and the local workstation Security Accounts Manager (SAM) database. The mathematical construction of the NT hash is remarkably basic:

$$\text{NTHash}=\text{MD4}(\text{UTF-16LE}(\text{plaintext\_password}))$$

```
[Plaintext Password] -> [Convert to UTF-16 Little Endian] ->
[Single Pass of MD4] -> [16-Byte NT Hash]
```

The string is encoded into 2-byte-per-character Unicode (UTF-16 Little Endian) and passed through a single execution pass of the Message Digest 4 (MD4) hashing algorithm. This design suffers from severe cryptographic engineering deficiencies:

Zero Salt

The algorithm applies no salt. The exact same password always yields the identical NT hash across every account, machine, and domain forest globally. This lack of uniqueness permits attackers to use precomputed lookup structures, such as global rainbow tables, to translate captured hashes back to plaintext passwords instantly.

NO KEY STRETCHING

The iteration count is exactly one (1). Modern key derivation functions (like PBKDF2, `bcrypt`, or Argon2) enforce thousands of hashing iterations to intentionally increase the CPU/GPU cycle cost of verifying a single password guess. Because MD4 is highly optimized for raw hardware execution speed, a modern commercial GPU array can calculate billions of MD4 operations per second, making brute-force verification highly efficient.

THE HASH IS THE STATIC SECRET

The NT hash functions as the primary key material for NTLM authentication and Kerberos Rivest Cipher 4 (RC4) ticket encryption. Because the wire protocol never performs a zero-knowledge proof of the password itself, an attacker who extracts the raw 16-byte NT hash does not need to crack it to gain access. They can simply present the hash directly to any network service that accepts NTLM, a technique known as a Pass-the-Hash (PtH) attack.

5.3.b NTLMv1 PROTOCOL MECHANICS

The NTLMv1 protocol relies on a three-way handshake (`NEGOTIATE`, `CHALLENGE`, `AUTHENTICATE`) to establish identity over a transport channel.

`THREE-WAY HANDSHAKE DIAGRAM`

TYPE 1 (`NEGOTIATE_MESSAGE`): The client initiates a connection to the target server and transmits a payload detailing its supported dialect capabilities, security flags, and domain context.

TYPE 2 (`CHALLENGE_MESSAGE`): The server processes the negotiation and generates a random, 8-byte nonce value known as the Server Challenge. This value is transmitted back to the client in plaintext over the wire.

TYPE 3 (`AUTHENTICATE_MESSAGE`): To prove identity, the client catches the 8-byte server challenge and performs the following calculation:

- The client takes its 16-byte NT hash and pads it with 5 trailing null bytes (`\x00`) to reach a total length of 21 bytes.
- This 21-byte string is cut into three distinct 7-byte blocks.
- Each 7-byte block is used as an independent DES encryption key.
- All three DES keys encrypt the identical 8-byte plaintext server challenge.
- The three resulting 8-byte ciphertexts are concatenated to form a final 24-byte payload knows as the NTLMv1 Response.

`TECHNICAL FLAWS`

Because the 21-byte padded hash uses 5 bytes of fixed zeros at the end of the third block, the final DES key contains only 2 bytes (16 bits total) of true entropy. This structural flaw allows attackers to recover the last block of the key material almost instantaneously, significantly reducing the remaining keyspace required to crack the full NT hash offline.

`5.3.c NTLMv2 PROTOCOL MECHANICS`

NTLMv2 was developed to introduce cryptographic variable bindings to prevent the deterministic replay attacks inherent in NTLMv1. It changes how the final handshake response is computed over the wire.

`NTLMv2 HANDSHAKE RESPONSE PROCESS`

`DERIVE THE NTLMv2 MASTER HASH`

Derive the NTLMv2 Master Hash: The client converts both the target username and the Active Directory domain name to uppercase strings, concatenates them, and computes an HMAC-MD5 hash of that payload, using the user's raw 16-byte **NT hash** as the cryptographic key.

Construct the Client Challenge Blob: The client generates its own randomized 8-byte nonce (the **Client Challenge**) and packs it into a structured variable-length byte array called the **Blob**. This blob contains critical structural fields, including the current system timestamp (to prevent indefinite replay), the target server's Target Info block (SPN, domain details), and a set of channel security structures.

Compute the NetNTLMv2 Response: The client concatenates the 8-byte cleartext server challenge with the newly constructed client challenge blob. It then processes this combined payload through an HMAC-MD5 operation, using the **NTLMv2 Master Hash** derived in Step 1 as the key. The resulting output is the **NetNTLMv2 Response**.

5.3.d THE ARCHITECTURAL BOUNDARY: NT HASH VS. NETNTLMv2

Confusing the NT hash with a NetNTLM response is a dangerous operational error for security professionals. They represent entirely different cryptographic structures with unique security boundaries.

Table 5.1 below compares and contrasts the differences between NT hashes and NetNTLM responses.

Dimension	The NT Hash (NTHash)	The NetNTLMv2 Response (Handshake)
Origin / Source	Extracted directly from memory (LSASS) or database files (`ntds.dit`, SAM)	Intercepted off the network wire during a live challenge-response exchange
Persistence	Stored statically and persistently inside operating system credential hives	Transient; exists only as a single exchange flight over the transport network
Pass-the-Hash (PtH) Use	**Yes.** Can be used directly as an authentication token to impersonate a user	**No.** Cannot be passed; it is bound to a specific session nonce
Offensive Utility	Direct access or fast offline brute-force cracking	Real-time session relay or slow offline cryptographic dictionary cracking

Hashcat Modes	Mode `1000` (NT) or Mode `3000` (LM)	Mode `5600` (NetNTLMv2)

5.3.1 NT HASH CONSTRUCTION AND CRYPTOGRAPHIC DEFICIENCIES

New Technology LAN Manager (NTLM) suite was introduced with Windows NT 3.1 in 1993 to rectify the structural vulnerabilities of LM. Its hash function is substantially better. The NT hash (commonly termed the `NTHash`) serves as the core authentication secret stored in the Active Directory database (`ntds.dit`) and the local workstation Security Accounts Manager (SAM) database. The mathematical construction of the NT hash is remarkably basic and is computed by taking the user's password, encoding it in Unicode (UTF-16 Little Endian), and computing the MD4 digest of the result.

That's it. One pass of MD4. No salt. No iteration count. No work factor.

$$\text{NTHash}=\text{MD4}(\text{UTF-16LE}(\text{plaintext\_password}))$$

```
[Plaintext Password] -> [Convert to UTF-16 Little Endian] ->
[Single Pass of MD4] -> [16-Byte NT Hash]
```

The string is encoded into 2-byte-per-character

The MD4 algorithm is fast by design. On modern hardware, billions of MD4 operations can be performed per second per GPU. Without a salt, a precomputed rainbow table can map hash values back to passwords with no computation at the time of attack. Without an iteration count or key derivation function adding cost to each guess, brute-force attacks proceed at the raw speed of the hardware.

This matters enormously for understanding Pass-the-Hash (PtH): the NT hash is not just a stored representation of the password. It is the actual key material that NTLM uses for authentication. An attacker who possesses the NT hash can authenticate as that user without ever knowing the plaintext password. The hash is the credential.

5.3.2 NTLMV1: THE ORIGINAL CHALLENGE-RESPONSE

NTLMv1 is the first iteration of the NTLM challenge-response protocol. The exchange works as follows:

Message 1-

`NEGOTIATE_MESSAGE` (Type 1): The client sends a negotiate message to the server advertising its capabilities and supported features.

Message 2-

`CHALLENGE_MESSAGE` (Type 2): The server responds with an 8-byte random value called the server challenge. This challenge is unique per authentication attempt and is sent in plaintext.

Message 3-

`AUTHENTICATE_MESSAGE` (Type 3): The client then compares the response. For NTLMv1, this is done by padding the NT hash to 21 bytes and splitting it into three 7-byte DES keys, each of which encrypts the 8-byte server challenge. The three resulting 8-byte values are concatenated to produce the 24-byte NTLMv1 response.

The Verification: The server (or a domain controller the server forwards to) performs the same computation against the stored NT hash and verifies that the response matches.

The Fundamental Problem

The entire security of NTLMv1 rests on the secrecy of the NT hash. Because the server challenge is sent in plaintext and the response is a deterministic function of the NT hash, anyone who captures the challenge and response pair and knows or can guess the NT hash can verify it offline. More dangerously, they can relay it – take the server's challenge and the client's response and forward the same exchange to a different server to authenticate as the user there, even without knowing the original password.

NTLMv1 also has a known weakness where the three DES keys derived from the 21-byte padded hash includes two bytes of zeros at the end of the third key, effectively reducing the search space for that portion and enabling optimized offline attacks.

5.3.3 NTLMv2: IMPROVEMENTS AND REMAINING WEAKNESSES

NTLMv2 was introduced to address the most egregious NTLMv1 weaknesses. It is the version that should be used in any environment that has not completely eliminated NTLM entirely, and it is what modern Windows systems default to.

The NTLMv2 response is computed a bit differently, as shown below:

STEP 1: DERIVE NTLMv2 HASH

Compute HMAC-MD5 of the server challenge, using as the key the HMAC-MD5 of the uppercase username concatenated with the uppercase domain name, keyed with the NT hash.

STEP 2: GENERATE CLIENT CHALLENGE

The client next generates its own random 8-byte value called the client challenge, or "blob." This is a crucial addition absent from NTLMv1.

STEP 3: COMPUTE NETNTLMv2 RESPONSE

Compute HMAC-MD5 over a value consisting of the server challenge, the client challenge, a timestamp, and other fields, keyed with the NTLMv2 hash from Step 1.

The resulting value is what most people call the NetNTLMv2 hash, though that term is technically imprecise. It is more accurately described as a NetNTLMv2 **response** or **handshake value**.

The client challenge addition was designed to prevent relay attacks by ensuring the response is unique and tied to both parties' randomized values; however, relay attacks against NTLMv2 remain possible in many real-world conditions – when the target server does not enforce channel signing or channel binding, the relay can still succeed because the authentication itself validates.

5.3.4 THE CRITICAL DISTINCTION: NT HASH vs. NETNTLMv2

This distinction is misunderstood constantly, including in published security materials, so it deserves its own section.

The NT hash is stored in the Security Accounts Manager (SAM) database, or `ntds.dit`. It can be extracted by dumping the database. It can be used directly to authenticate – presented in place of a password. This is the Pass-the-Hash (PtH) attack. The attacker needs no knowledge of the plaintext password; the hash is now the credential.

The NetNTLMv2 response is capture on the wire during an NTLM authentication exchange – most commonly by a tool like Responder or Inveigh that answers LLMNR or NBT-NS queries and captures the resulting authentication attempt. It is not stored anywhere persistently. It cannot be used directly to authenticate (you cannot "pass" it). It can be cracked offline using a dictionary attack against the underlying NT hash, or it can be relayed in real-time to authenticate to a different server that accepts NTLM.

To summarize the differences:

- NT hash → extract from SAM/ntds.dit → Pass-the-Hash (PtH) → direct authentication
- NetNTLMv2 → capture on the wire → crack or relay → no direct use

Tools like Hashcat handle both, but with different mode numbers: mode `5600` for NetNTLMv2 (cracking), modes `1000` or `3000` for NT/LM hashes. The failure to distinguish between them leads to defenders monitoring for the wrong thing and attackers wasting their time on the wrong technique.

5.3.5 WHERE NTLM IS STILL USED

Kerberos is the preferred authentication protocol for domain authentication, and Windows will use it whenever possible. But NTLM is still invoked in a significant set of persistent conditions:

- Authentication to a server using an IP address rather than a hostname (Kerberos requires a hostname to look up the accompanying SPN)

- Authentication to local accounts on domain-joined machines (local accounts have no Kerberos principal)
- Authentication to systems not joined to a domain
- Authentication when the KDC is unreachable
- Applications explicitly requesting NTLM via SSPI flags
- File share and SMB connections in certain configurations
- Older applications that do not support Kerberos

Because NTLM falls back automatically in many of these conditions, environments that believe they have restricted NTLM frequently discover it still flowing when any of these conditions are triggered.

5.3.6 OFFENSIVE IMPLICATIONS OF NTLM (CONCEPTUAL OVERVIEW)

The protocol mechanics described above create several categories of attack that will be fully examined in Part II. Understanding them at a conceptual level now is appropriate because the attacks are direct consequences of the protocol design.

5.3.6.1 PASS-THE-HASH (PtH)

Because the NT hash is the actual authentication material, an attacker who extracts it from memory or the credential store can authenticate as the user without knowing their password. This attack targets any service that accepts NTLM authentication, including remote file shares, Windows Management Instrumentation (WMI), Remote Procedure Call (RPC) services, and local administrative interfaces.

5.3.6.2 NTLM RELAY

Because the server challenge is sent in plaintext and the client's response is a function of it, an attacker who can intercept the exchange and forward it to a different target before it expires can authenticate to that target as the victim. Extended Protection for Authentication (EPA) and SMB signing both exist specifically to prevent this; an environment without both is vulnerable to relay in various scenarios.

5.3.6.3 CREDENTIAL CAPTURE

Tools that answer broadcast name-resolution queries (LLMNR, NBT-NS, MDNS) can trick Windows clients into performing NTLM authentication against the attacker's machine, capturing the NetNTLMv2 responses for later offline cracking.

5.3.7 DEFENSIVE CONTROLS FOR NTLM

NTLMv1 MUST BE DISABLED

There is no justifiable reason to accept NTLMv1 in a modern environment.

> **Group Policy:** `Computer Configuration → Windows Settings → Security Settings → Local Policies → Security Options →` *"Network security: LAN Manager authentication level"* should be set to *"Send NTLMv2 response only. Refuse LM & NTLM."*

NTLM AUDITING BEFORE RESTRICTION

Before disabling NTLM, configure audit logging to understand what is using it.

> **Group Policy:** *"Network security: Restrict NTLM: Audit NTLM authentication in this domain."*

This setting enables Event IDs 8001–8004 in the Microsoft-Windows-NTLM operational log, providing visibility into every NTLM authentication attempt including source, destination, and username.

SMB SIGNING

SMB signing prevents an attacker from modifying SMB traffic in transit. More directly relevant to relay attacks: a client that requires signing will reject an SMB server that does not sign, breaking the relay chain.

> **Group Policy:** *"Microsoft network client: Digitally sign communications (always)"* and *"Microsoft network server: Digitally sign communications (always)."*

Extended Protection for Authentication (EPA) / Channel Binding. EPA binds NTLM authentication to the underlying TLS channel, preventing an attacker from relaying credentials captured on one channel to a different channel. Required for NTLM relay to LDAP to be truly mitigated; configure via "LDAP signing" and "LDAP channel binding" Group Policy settings.

Monitoring signals: Event ID 4776 (credential validation on the local machine) and 4624 with Logon Type 3 using NTLM authentication package are the primary signals. A Kerberos-first domain where NTLM events spike is producing an anomaly worth investigating.

5.4 KERBEROS: THE ARCHITECTURE OF DELEGATED TRUST

Where NTLM is a challenge-response protocol built around a stored hash, Kerberos is a ticket-based system built around the principle that credentials should never leave the client machine. It is the primary authentication protocol for Active Directory, preferred over NTLM wherever both are available, and it is the protocol whose internals underlie the most consequential Active Directory attacks — Kerberoasting, Golden Tickets, Silver Tickets, and delegation abuse all operate by manipulating Kerberos mechanics rather than bypassing them. Understanding Kerberos correctly means understanding it as a trust architecture: a system that issues cryptographically sealed vouchers of identity, delegates their use across service boundaries, and embeds Windows authorization data inside them so that access control decisions can be made

locally without repeated trips to the domain controller. The subsections that follow trace that architecture from its origins and key components through the full five-message authentication sequence, the Privilege Attribute Certificate, encryption types, Service Principal Names, and delegation models.

5.4.1 ORIGINS AND DESIGN PHILOSOPHY

Kerberos was developed at the Massachusetts Institute of Technology (MIT) as part of Project Athena in the late 1980s, published as an open specification, and adopted by Microsoft as the primary authentication protocol for Windows 2000 Active Directory and all subsequent versions. The current standard is Kerberos version 5, defined in RFC 4120.

The design goals of Kerberos were specific and deliberate, shaped by the threat model of its era and by the fundamental insight that passwords should never traverse the network in any form Three principles guide the entire protocol:

1. **MUTUAL AUTHENTICATION**
   Both the client and the server verify each other's identity. A client should not send sensitive data to a server it hasn't authenticated; a server should not grant access to a client it hasn't authenticated; a server should not grant access to a client it hasn't authenticated.
2. **NO PLAINTEXT CREDENTIALS ON THE WIRE**
   The password never leaves the client. What leaves the client is a cryptographic proof derived from the password — a proof that is tied to a specific time, a specific server, and a specific session, rendering it useless to a passive observer.

3. **Ticket-based, time-limited authorization.** Rather than verifying identity from scratch on every service access, Kerberos issues tickets — cryptographic objects that carry identity and authorization information — that clients can present to services within a time window.

Microsoft implemented Kerberos with Active Directory and extended it with the Privileged Attribute Certificate (PAC), which carries the Windows-specific authorization data (group memberships, and Security Identifiers (SIDs)) that Active Directory's access control model depends on.

5.4.2 KEY COMPONENTS

The Key Distribution Center (KDC) runs as a service on every domain controller. It is logically divided into two components that share a database but perform distinct functions:

- **Authentication Service (AS):** Processes AS-REQ messages from clients seeking initial authentication. Issues Ticket Granting Tickets.

- **Ticket Granting Service (TGS):** Processes TGS-REQ messages from clients seeking access to specific services. Issues service tickets.

The krbtgt account is a special, built-in domain account that exists solely to be the KDC's signing identity. Its password-derived key encrypts every Ticket Granting Ticket issued in the domain. Compromise of this ke— or knowledge of it — enables an attacker to forge TGTs for any principal, with any group memberships, for any duration. This is the Golden Ticket attack. The krbtgt account's password is never set manually and never used interactively; it is managed entirely by the KDC.

Principals are the identities that participate in Kerberos. In Windows Active Directory, principals include users, computers, and service accounts. Each principal has a long-term key derived from its password.

The realm in RFC 4120 terms corresponds to the Active Directory domain. Cross-realm authentication (cross-domain or cross-forest) uses trust relationships between KDCs.

Ticket Granting Tickets (TGTs) are issued by the AS. They are encrypted with the krbtgt key, which means the client cannot read the contents of its own TGT. The client simply holds the TGT and presents it to the TGS when requesting service tickets. Because the client cannot decrypt or modify the TGT without the krbtgt key, the TGT is tamper-resistant from the client's perspective.

Service tickets (formally called service tickets or TGS-REP tickets) are issued by the TGS. They are encrypted with the service account's long-term key, which means only the target service can read them. The client presents the service ticket to the service to prove its identity.

5.4.3 Kerberos Pre-Authentication

Before examining the full authentication flow, pre-authentication deserves its own explanation because its presence or absence is the difference between two distinct attack paths.

With pre-authentication enabled (normal, default): When a client sends an AS-REQ requesting a TGT, it must include a PA-DATA field containing a PA-ENC-TIMESTAMP structure — the current timestamp, encrypted with the client's long-term key. This proves to the KDC that the requester knows the client's key material. The KDC decrypts the timestamp, verifies it is within the clock skew tolerance (default five minutes), and only then issues the AS-REP.

Without pre-authentication (DONT_REQ_PREAUTH flag set): The KDC will issue an AS-REP to anyone who requests it for that principal, without requiring any proof that the requester knows the client's key. The encrypted portion of the AS-REP — which is sealed

with the client's long-term key — is returned to an unauthenticated requester. That encrypted blob can be taken offline and subjected to dictionary attack to recover the plaintext password.

This is AS-REP Roasting. The attack works entirely because the KDC will respond to an unauthenticated request. The DONT_REQ_PREAUTH flag exists for legacy application compatibility and should never be set on active accounts in a modern environment.

The userAccountControl attribute bit for this flag is 0x400000 (decimal 4194304). Query:

Get-ADUser -Filter 'useraccountcontrol -band 4194304' -Properties userAccountControl

5.5 THE KERBEROS AUTHENTICATION FLOW: STEP BY STEP

A complete Kerberos authentication from a cold start — no cached tickets, no prior sessions — involves five distinct messages exchanged across two separate connections: first between the client and the Key Distribution Center (KDC), then between the client and the target service. Each message has a defined structure, specific encrypted and cleartext components, and a precise security purpose. In my experience assessing Active Directory environments, the most common gap in a practitioner's Kerberos knowledge is not ignorance of the individual messages but failure to understand which party can read which portion of each message — and therefore which party an attacker must compromise to forge or abuse it. The subsections below walk each message in sequence, with that question held consistently in focus.

5.5.1 STEP 1 - AS-REQ: THE INITIAL AUTHENTICATION REQUEST

The client begins authentication by sending an Authentication Service Request (AS-REQ) to the KDC. This message contains:

- **cname:** The client's principal name (the username)
- **realm:** The domain name
- **sname:** The requested server — for initial authentication, this is always krbtgt/DOMAIN (the TGS itself)
- **till:** The requested ticket expiration time
- **nonce:** A random number to prevent replay
- **etype:** The list of encryption types the client supports, in order of preference
- **PA-DATA:** The pre-authentication data (the encrypted timestamp, when required)

The timestamp encrypted in PA-DATA is sealed with the client's long-term key — in Windows, this is typically the AES256 key derived from the password via the Kerberos AES string-to-key function. The KDC derives the same key from the stored verifier and uses it to decrypt and validate the timestamp.

Clock synchronization dependency: This is where time synchronization becomes a hard cryptographic requirement. If the client's clock differs from the KDC's clock by more than the default five-minute window (defined in the domain's Kerberos policy), the KDC will reject the AS-REQ with KRB_AP_ERR_SKEW. There is no workaround and no fallback. Authentication fails. This is why chapter 3's discussion of time synchronization as a hard dependency was not academic — it is enforced in the protocol itself.

5.5.2 STEP 2 — AS-REP: THE TGT IS ISSUED

If the AS-REQ validates, the KDC returns an Authentication Service Reply (AS-REP) containing two components:

Component 1 — The Ticket Granting Ticket (TGT): Encrypted with the krbtgt account's long-term key. The client cannot read this. Its contents include:

- The client's principal name
- The session key (a freshly generated symmetric key for this authentication session)
- The ticket start time, end time, and renewal limit
- Ticket flags (renewable, forwardable, proxiable, etc.)
- The client's IP address (optional)
- The Privilege Attribute Certificate (PAC) — covered in section 5.7

Component 2 — The encrypted reply part: Encrypted with the client's long-term key. The client decrypts this to obtain the TGT session key. Its contents include:

- The TGT session key
- The nonce from the AS-REQ (proves freshness)
- The ticket's validity window

After this exchange, the client holds: (a) the opaque TGT it cannot read, and (b) the TGT session key that proves it is the TGT's legitimate holder. Both are stored in LSASS memory on the client workstation.

5.5.3 STEP 3 - TGS-REQ: REQUESTING A SERVICE TICKET

When the client needs to access a service, it sends a Ticket Granting Service Request (TGS-REQ) to the KDC. This message contains:

- **The TGT:** Presented opaquely to the KDC

- **The Authenticator:** A structure containing the client principal name and current timestamp, encrypted with the TGT session key. This proves the client possesses the session key and is therefore the legitimate TGT holder.
- **sname:** The Service Principal Name of the target service
- **nonce:** A fresh random value
- **etype:** Supported encryption types (ordered by preference)
- **Additional tickets:** If requesting constrained delegation

The KDC decrypts the TGT using the krbtgt key, extracts the TGT session key, and uses it to decrypt the Authenticator. If the Authenticator's timestamp is current and the Authenticator has not been seen before (replay prevention), the KDC processes the request.

This exchange is where Kerberoasting lives. The service ticket requested here will be encrypted with the target service account's long-term key. Any authenticated domain user can make this request for any SPN-registered account. The KDC does not check whether the requester should be permitted to use the service — ticket issuance is unconditional. An attacker who extracts this ticket and takes it offline can mount a dictionary attack against the service account's password without any further interaction with the domain. The only prerequisite is a valid domain account and knowledge of a target SPN.

5.5.4 STEP 4 - TGS-REP: THE SERVICE TICKET IS ISSUED

The KDC returns a Ticket Granting Service Reply (TGS-REP) containing:

Component 1 — The service ticket: Encrypted with the target service account's long-term key. The client cannot read this. Its contents include:

- The client's principal name
- A new session key (for the client-service communication)
- The ticket's validity window
- The PAC, copied from the TGT and re-signed for this service

Component 2 — The encrypted reply part: Encrypted with the TGT session key. The client decrypts this to obtain the service session key.

The default ticket lifetime is 10 hours. Tickets are renewable up to a maximum lifetime, typically seven days.

5.5.5 STEP 5 - AP-REQ: AUTHENTICATING TO THE SERVICE

The client sends an Application Request (AP-REQ) to the target service. This message contains:

- **The service ticket:** The opaque blob from the TGS-REP
- **The Authenticator:** Client name and timestamp, now encrypted with the service session key

The service decrypts the service ticket using its own long-term key, extracts the service session key, and uses it to decrypt the Authenticator. If the Authenticator validates, the client is authenticated.

Optional mutual authentication (AP-REP): If the client requested mutual authentication, the service returns an AP-REP encrypted with the service session key, proving to the client that the service also holds the key — confirming it is the legitimate service and not an impersonator.

The service does not contact the KDC to validate the ticket. It trusts the KDC's signature — the fact that the ticket is correctly encrypted with the service account's key is the proof of legitimacy. This offline verification property is what makes Silver Ticket attacks possible: an attacker who knows the service account's NT hash can forge a service ticket entirely offline, without any KDC interaction, and present it directly to the service.

5.6 ENCRYPTION TYPES IN KERBEROS

Encryption type selection is not merely a configuration preference. It directly determines the cryptographic strength of every ticket and the cost of offline recovery if one is captured.

5.6.1 THE ENCRYPTION TYPES

Active Directory supports four Kerberos encryption types across its history, ranging from the completely broken to the currently recommended. Each encryption type appears in ticket requests and responses as a numeric identifier called an etype, and the etype present in a captured ticket directly determines whether offline recovery is practical and how long it takes. When I have reviewed environments flagged for Kerberoasting exposure, the etype field in Event ID 4769 is the first thing I examine — it tells me immediately whether the attacker has a fast path to credential recovery or a slow one. The four types, from weakest to strongest, are as follows.

5.6.1.1 DES (etype 1 and 3)

Disabled by default since Windows Server 2008. When found in a modern environment, it indicates a seriously outdated configuration. DES keys are 56 bits and are computationally trivial to crack.

5.6.1.2 RC4-HMAC (etype 23)

The most important encryption type from an offensive standpoint. The RC4 key used for Kerberos encryption is derived directly from the account's NT hash — with no additional derivation function, no salting, and no key stretching. This means:

- For Kerberoasting: a TGS-REP ticket encrypted with RC4 is keyed with the service account's NT hash. Hashcat mode 13100 can test billions of candidates per second on modern hardware.
- For AS-REP Roasting: an AS-REP encrypted with RC4 is keyed with the user account's NT hash. Hashcat mode 18200.
- For Golden and Silver Tickets: an attacker who knows the NT hash of krbtgt or a service account can immediately forge RC4-encrypted tickets without additional computation.

RC4 remains supported in most environments for backward compatibility. It is the attacker's preferred encryption type because it is the fastest to crack.

5.6.1.3 AES128-CTS-HMAC-SHA1-96 (etype 17)

AES with a 128-bit key. Keys are derived using the AES string-to-key function (RFC 3962), which involves PBKDF2 with the password, a salt, and an iteration count. This is orders of magnitude more expensive than RC4 derivation. Cracking AES-encrypted Kerberos material is feasible for weak passwords but dramatically slower than RC4.

5.6.1.4 AES256-CTS-HMAC-SHA1-96 (etype 18)

AES with a 256-bit key. The preferred and most secure option. The key derivation is the same PBKDF2-based function as AES128 but with a longer key. Enforcing AES256 is the single most effective cryptographic control against offline Kerberos ticket cracking.

5.6.2 THE `msDS-SupportedEncryptionTypes` ATTRIBUTE

Each Active Directory account has an attribute called msDS-SupportedEncryptionTypes that declares which encryption types the KDC will use when issuing tickets for or to that account. When this attribute is zero or absent, the KDC defaults to RC4 for backward compatibility. Setting this attribute to 24 (the bitmask for AES128 + AES256) tells the KDC to use AES and not offer RC4.

powershell

Enforce AES128 and AES256, remove RC4 support for a specific account

Set-ADUser -Identity "svc-sqlreport" -KerberosEncryptionType AES128,AES256

Verify

Get-ADUser "svc-sqlreport" -Properties msDS-SupportedEncryptionTypes |

Select msDS-SupportedEncryptionTypes

Setting this attribute to 24 means a Kerberoasted ticket for that account will be AES-encrypted. For an account with a strong, random password, AES-encrypted tickets are computationally impractical to crack. For an account with a gMSA (240-byte random password rotated automatically), the encryption type is irrelevant — no password strength permits cracking.

5.6.3 ENCRYPTION TYPE DETECTION

Event ID 4769 (a Kerberos service ticket was requested) includes the ticket encryption type in its data. In a domain that enforces AES, a service ticket request specifying encryption type 0x17 (RC4) is an anomaly. It indicates either a legacy client that cannot negotiate AES or an attacker specifically requesting RC4 to enable offline cracking. Both scenarios warrant investigation.

5.7 THE PRIVILEGE ATTRIBUTE CERTIFICATE (PAC)

Standard Kerberos, as defined in RFC 4120, carries identity but not Windows authorization data. A service that receives a Kerberos ticket knows who the client is — their principal name — but has no direct knowledge of what groups they belong to, what their Security Identifier (SID) is, or what access permissions apply to them under Windows access control. Microsoft solved this problem by extending the Kerberos ticket format with the Privilege Attribute Certificate, embedding Windows-specific authorization data directly inside the ticket so that services can make access decisions locally without a separate directory query. The PAC is not a minor implementation detail. It is the structure that carries every group membership, every SID, and every authorization attribute that Windows access control depends on — and it is the structure whose integrity, or lack thereof, determines whether ticket forgery gives an attacker arbitrary access or merely authenticated access. The subsections below examine what the PAC contains, how its signatures protect it, and what happens when those protections fail.

5.7.1 THE SIGNIFICANCE OF THE DOMAIN PAC

The Privilege Attribute Certificate is a Microsoft-specific extension to Kerberos that does not exist in the RFC 4120 specification. Microsoft added it to solve a problem unique to Windows environments: Kerberos tickets, as defined in the standard, carry identity information but not Windows authorization data. A service that receives a Kerberos service ticket knows who the client is, but does not know what groups they belong to, what their Security Identifier (SID) is, or what access control entries apply to them.

The PAC embeds Windows authorization data directly into the ticket, allowing a service to make access control decisions locally without contacting the KDC or a domain controller for each request. This is the mechanism that makes Windows resource authorization based on group membership practical at scale.

5.7.2 PAC CONTENTS

The PAC is embedded in the authorization-data field of the ticket. It is a complex structure containing multiple buffers:

KERB_VALIDATION_INFO: The core authorization buffer. Contains:

- The user's logon name and domain name
- The user account's SID (the UserSid field)
- The Primary Group SID
- All group SIDs the user is a member of, both domain local and global
- SID history (for migrated accounts)
- User account flags and control settings
- Password last set, account expiration, and logon count information
- The user's profile path and home directory information
- The logon server and logon domain names

PAC_CLIENT_INFO: Simplified client identification containing the client name and authentication time.

UPN_DNS_INFO: The User Principal Name and DNS domain name.

PAC_SERVER_CHECKSUM: An HMAC-MD5 checksum of the entire PAC, keyed with the service account's long-term key. The target service verifies this checksum to detect tampering.

PAC_PRIVSVR_CHECKSUM: An HMAC-MD5 checksum of the entire PAC, keyed with the krbtgt key. Only the KDC can produce or verify this signature. It is the primary integrity control.

5.7.3 PAC VALIDATION

When a service receives a service ticket, it may optionally request that the KDC validate the PAC. This is done through the KERB_VERIFY_PAC request. The KDC verifies the PAC_PRIVSVR_CHECKSUM and confirms that the PAC content matches what it would have issued. If the checksums match, the PAC is genuine.

Many services do not call KERB_VERIFY_PAC — they simply check the PAC_SERVER_CHECKSUM, which they can verify locally using their own key, and accept the authorization data as trustworthy because the ticket itself was correctly decrypted. This is fine in normal operation but creates a subtle attack surface.

5.7.4 PAC and the Privilege Escalation History

In November 2014, Microsoft patched CVE-2014-3566, commonly known as MS14-068. The vulnerability was in the KDC's handling of privilege attribute certificate validation: a low-privileged domain user could submit a forged PAC to the KDC and receive a legitimate, KDC-signed ticket embedding the forged authorization data — including membership in Domain Admins. The KDC signed what it was handed rather than generating the PAC from the authoritative account data.

MS14-068 is patched and has been for over a decade. It is mentioned here not as an active vulnerability but because it illustrates why the PAC matters: whoever controls the PAC controls what groups the service believes the user belongs to, and group membership is the basis of Windows access control. The PAC signature chain — user's long-term key, service key, and krbtgt key — is the mechanism that keeps a client from writing its own authorization data.

5.7.5 GOLDEN TICKETS REVISITED THROUGH THE PAC LENS

A Golden Ticket is not merely a forged TGT. It is a forged TGT with a forged PAC that claims arbitrary group memberships — including Domain Admins, Enterprise Admins, or any other group — signed with the krbtgt key. When an attacker knows the krbtgt NT hash, they can compute the PAC_PRIVSVR_CHECKSUM themselves, produce a ticket that passes all KDC and service validation, and present it to any service in the domain. The ticket is indistinguishable from a legitimate one because the cryptographic signatures are genuine — just made with the attacker's copy of the key, not the KDC's.

5.8 SERVICE PRINCIPAL NAMES

Kerberos needs a way to connect a client’s request for a specific service to the account running that service, so the Key Distribution Center (KDC) can retrieve the correct key to encrypt the resulting service ticket. Service Principal Names (SPNs) are that connection mechanism – a structured naming convention that registers services to accounts in the directory, making the KDCs lookup deterministic and ensuring the right key is used for every ticket. In my penetration testing experience, SPN enumeration is one of the first things a skilled operator does after obtaining any solid domain foothold, because the SPN list is effectively a map of every Kerberoastable target in the environment. Understanding what SPNs are, how they are structured, and what their registration implies about the security posture of the accounts they point to is foundational knowledge for both attackers and defenders alike.

5.8.1 SPN AND ITS ROLE AS A DOMAIN IDENTIFIER CONNECTOR

A Service Principal Name (SPN) is the identifier that connects a Kerberos service request to the account running the service. When a client sends a TGS-REQ, it names the target service

using the service's SPN. The KDC queries the directory to find which account has that SPN registered, retrieves that account's long-term key, and uses it to encrypt the service ticket.

SPNs follow the format:

serviceClass/host

serviceClass/host:port

serviceClass/host:port/serviceName

Examples:

- MSSQLSvc/sql01.ficam.local:1433 — SQL Server
- HTTP/webserver.ficam.local — IIS/HTTP service
- HOST/workstation01.ficam.local — general host SPN
- CIFS/fileserver.ficam.local — SMB file sharing

SPNs are stored in the servicePrincipalName attribute of the account running the service. When a service starts on a domain-joined computer, it typically registers its SPNs automatically. Service accounts for custom applications have SPNs registered by administrators.

5.8.2 SPNs AND KERBEROASTING

The SPN is the mechanism that makes Kerberoasting work. Any SPN registered to any account creates a requestable service ticket encrypted with that account's key. There is no authorization check. There is no requirement that the requester should be permitted to use the service. The KDC issues the ticket to any authenticated domain principal that asks.

This design is by specification — Kerberos service ticket issuance is authorization-neutral. The service is responsible for its own access control once it decrypts the ticket. But the implication is that every SPN registration is simultaneously a declaration of service availability and the publication of encrypted material tied to a crackable key.

5.8.3 DUPLICATE SPNs

A less-commonly discussed problem: duplicate SPNs. If the same SPN is registered to two different accounts in the domain, Kerberos does not know which account's key to use. The KDC cannot resolve the ambiguity deterministically and authentication to that service will fail. Beyond the availability impact, duplicate SPNs indicate inconsistent account management — the same kind of environment where Kerberoastable accounts accumulate unnoticed.

Identify duplicates:

cmd

setspn -X -F

5.9 KERBEROS DELEGATION

Delegation is the mechanism by which a service can authenticate to another service on behalf of a user. The canonical use case is a web application that needs to connect to a SQL database as the authenticated user, not as the web application's service account. Kerberos provides three delegation models, each with distinct security properties.

5.9.1 UNCONSTRAINED DELEGATION

What it is: When an account is configured for unconstrained delegation (the "Trust this computer for delegation to any service (Kerberos only)" flag in Active Directory), the KDC includes the user's full TGT inside the service ticket delivered to the delegating service. When a user authenticates to that service, the service extracts the TGT from the ticket and stores it in its own LSASS process. It can then use that TGT to request service tickets for any service, as the user, with no restrictions.

Why this is dangerous: Any domain controller that authenticates against an unconstrained delegation host delivers its TGT to that host. In environments where domain controllers perform Kerberos authentication against these hosts — which happens regularly during various administrative operations — an attacker who has compromised the unconstrained delegation host can extract those domain controller TGTs from LSASS. This is the foundation of several attack chains, including PrinterBug (MS-RPRN) coercion and PetitPotam (MS-EFSRPC) coercion, where an attacker forces a domain controller to authenticate to the compromised host, then extracts the resulting TGT.

Identifying unconstrained delegation accounts:

powershell

Get-ADComputer -Filter {TrustedForDelegation -eq $true} -Properties TrustedForDelegation

Get-ADUser -Filter {TrustedForDelegation -eq $true} -Properties TrustedForDelegation

5.9.2 CONSTRAINED DELEGATION (S4U2Proxy)

What it is: Constrained delegation was introduced to limit unconstrained delegation's blast radius. An account configured for constrained delegation can only delegate to a specific list of services, enumerated in the msDS-AllowedToDelegateTo attribute. The delegation is enforced by the KDC rather than relying on the service itself.

5.9.3 THE S4U EXTENSIONS

Constrained delegation relies on two Service-for-User (S4U) protocol extensions:

- **S4U2Self:** Allows a service to request a service ticket to itself on behalf of an arbitrary user, producing a forwardable ticket even if the user did not originally obtain one. This is sometimes useful for legitimate services but creates a path for privilege escalation when combined with S4U2Proxy.
- **S4U2Proxy:** Allows a service to request a service ticket to another service on behalf of a user, but only for services listed in msDS-AllowedToDelegateTo.

Offensive implication: An account configured for constrained delegation with S4U2Self can impersonate any domain user to itself, then use S4U2Proxy to access any service in its delegation list as that user — including domain administrators. The constraint is the list, and attackers who can modify msDS-AllowedToDelegateTo on an account can extend its reach.

5.9.4 RESOURCE-BASED CONSTRAINED DELEGATION (RBCD)

What it is: Introduced with Windows Server 2012, Resource-Based Constrained Delegation inverts the configuration model. Rather than configuring which services the delegating account can delegate to, RBCD is configured on the resource itself: the resource's `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute specifies which principals may delegate to it.

This inversion has a significant security implication: any principal with write access to a computer's `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute - which includes the computer itself in many default configurations - can configure RBCD to allow delegation to it from an account the attacker controls. This creates an attack path where an attacker who can write to this attribute on a high-value system can combine it with S4U2Self and S4U2Proxy to authenticate to that system as any user, including Domain Admins.

RBCD is one of the most frequently abused privilege escalation paths in modern Active Directory environments because the prerequisite (write access to the attribute) is surprisingly common and because the attack does not require any account already configured for delegation.

5.10 KERBEROS ARMORING AND PROTECTED USERS

The base Kerberos protocol, as described in the preceding sections, has two significant exposure points that additional controls are designed to close. The first is the pre-authentication exchange itself: without additional protection, a captured AS-REQ can be subjected to offline dictionary attack if the DON’T_REQ_PREAUTH` flag is absent but pre-authentication data is observable. The second is the absence of any default restriction on how domain accounts authenticate – unless explicitly constrained, accounts can use NTLM, RC4, unconstrained delegation, and long-lived tickets, each of which expands the attack surface. Kerberos Armoring and the Protected Users security group address both exposure points through different mechanisms: Armoring

encrypts the pre-authentication exchange, while Protected Users imposes non-negotiable authentication restrictions at the KDC level regardless of client or application configuration.

5.10.1 KERBEROS FAST (FLEXIBLE AUTHENTICATION SECURE TUNNELING)

Kerberos Armoring, specified in RFC 6113 and branded by Microsoft as FAST (Flexible Authentication Secure Tunneling), protects the pre-authentication exchange from offline attack by wrapping it in an encrypted tunnel keyed with a machine account ticket. In an armored exchange, the `PA-ENC-TIMESTAMP` is not visible in the clear to an observer on the wire. An attacker who captures the AS-REQ cannot mount an offline dictionary attack against the timestamp because the timestamp itself is protected.

FAST requires:

- Domain functional level of Windows Server 2012 or higher
- "Support Dynamic Access Control and Kerberos armoring" Group Policy enabled on both clients and domain controllers
- Claims, compound authentication, and Kerberos armoring supported on the target

Not all environments have FAST deployed. Where it is absent, AS-REQ pre-authentication data is observable and the offline attack against it is straightforward, though it still requires the `DONT_REQ_PREAUTH` flag to be set for AS-REP Roasting.

5.10.2 THE PROTECTED USERS SECURITY GROUP

The Protected Users group, introduced in Windows Server 2012 R2, is a special security group whose members are subject to a set of non-negotiable Kerberos restrictions enforced by the KDC itself - not by Group Policy, not by individual settings, not by a flag that can be accidentally cleared.

Members of Protected Users receive the following protections automatically:

- **No NTLM authentication.** Kerberos is required. NTLM fallback is blocked.
- **No DES or RC4 encryption.** AES is required. This alone eliminates Kerberoasting's fast offline path for any Protected Users member.
- **No unconstrained delegation.** Cannot be configured for unconstrained delegation and will not send their credentials to an unconstrained host.
- **No credential caching.** Cached credentials for Protected Users are not stored on endpoints, which means offline attacks against a disconnected machine cannot recover their credentials.
- **Reduced ticket lifetime.** TGT lifetime is limited to four hours (non-renewable), reducing the window in which a stolen TGT can be used.

Tier 0 accounts - domain admins, enterprise admins, `krbtgt` - should be members of Protected Users. Any account that should not be authenticated via NTLM or RC4 should be in Protected Users. The group enforces these restrictions at the KDC, which means they cannot be bypassed by client-side configuration errors.

> **Important caveat:** Adding accounts to Protected Users without first auditing which services and applications those accounts touch can break things. Applications that rely on NTLM or RC4 for accounts in this group will stop working. Test thoroughly before adding production service accounts or accounts used by legacy applications.

5.11 KERBEROS EVENT IDs AND MONITORING

Authentication protocol monitoring in Windows Active Directory is anchored on a set of Security event log Event IDs that record each step of the Kerberos and NTLM exchanges. Understanding what each event records - and what its absence means - is the foundation of identity telemetry.

Event ID 4768: A Kerberos authentication ticket (TGT) was requested.

- Generated on the domain controller when an AS-REQ is processed.
    - ***Key fields:***
        - Account Name (the requesting principal)
        - Result Code (`0x0` = success; `0x18` = wrong password; `0x6` = no such user)
        - Pre-Authentication Type (`0` = no pre-auth - the AS-REP Roasting signal)
        - Ticket Encryption Type (`0x17` = RC4; `0x12` = AES256).

Event ID 4769: A Kerberos service ticket was requested.

- Generated on the domain controller when a TGS-REQ is processed.
    - ***Key fields:***
        - Account Name (the requesting principal)
        - Service Name (the SPN requested)
        - Ticket Encryption Type (`0x17` = RC4 = Kerberoasting signal)
        - Failure Code. A burst of 4769 events for multiple different SPNs from a single account within a short window is the primary Kerberoasting detection.

Event ID 4771: Kerberos pre-authentication failed.

- Generated when an AS-REQ is submitted with incorrect pre-authentication data - effectively a failed password attempt using Kerberos.
    - ***Key fields:***
        - Client Address (source IP)

        - Failure Code (`0x18` = wrong password).

Event ID 4770: A Kerberos service ticket was renewed.

- Lower-value for detection but useful for understanding session patterns and ticket lifetime.

Event ID 4776: The domain controller attempted to validate the credentials of an account (NTLM).

- Generated on the domain controller for NTLM credential validation.
    - ***Key fields:***
        - Account Name
        - Workstation
        - Error Code (`0x0` = success; `0xC000006A` = wrong password).

Event ID 4624 / 4625: Logon succeeded / failed.

- Generated on the authenticating system. The `LogonType` and `AuthenticationPackageName` fields distinguish Kerberos from NTLM authentication, which is why these events are used in baseline detection for unexpected NTLM usage.

5.12 NTLM VERSUS KERBEROS: A DIRECT COMPARISON

Table 5.1 below provides a direct comparison of the two protocol families across the dimensions that matter most to security practitioners.

Table 5.1: *Comparison of NTLM and Kerberos authentication across security-relevant dimensions*

Dimension	NTLM	Kerberos
Authentication model	Challenge-response	Ticket-based
Credential on the wire	Challenge-response hash	Encrypted timestamp (pre-auth)
Mutual authentication	Not by default	Built-in (AP-REP)
Requires domain connectivity	No (can use local SAM)	Yes (KDC required)
Hash type used	NT hash (MD4)	AES or RC4 (etype-dependent)

Dimension	NTLM	Kerberos
Offline cracking risk	High (NT hash derivable from response)	Depends on etype; RC4 high, AES lower
Relay attack risk	High	Very low (PAC + ticket binding)
Pass-the-Hash possible	Yes	No (but Pass-the-Ticket is analogous)
Standard specification	Proprietary (Microsoft)	RFC 4120
Legacy status	Legacy; should be restricted	Primary; preferred
Authorization data	Basic (in token)	Full PAC (SIDs, groups)
Delegation support	No	Yes (unconstrained, constrained, RBCD)
Primary detection events	4776, 4624	4768, 4769, 4771
Protocol attack patterns	Relay, credential capture, PtH	Roasting, ticket forgery, delegation abuse

---

5.13 SUMMARY AND TRANSITION

This chapter has mapped the two authentication protocol families on which Active Directory depends.

In this chapter, you learned:

- NTLM is a legacy challenge-response protocol that is fast, adaptable, and comprehensively understood by attackers: its NT hash is the credential that can be passed directly, its Net-NTLMv2 response is a crackable or relayable artifact that confirms the distinction between the two is not semantic but operational.
- NTLMv1 must be disabled; NTLMv2 must be audited and progressively constrained.
- SMB signing and EPA close the relay path that NTLM's design inherently opens.

- Kerberos is the primary authentication protocol, more architecturally sound, more defensible, and carrying more authorization context - but not without attack surface.
- Pre-authentication, when disabled, creates AS-REP Roasting.
- Service ticket issuance, being unconditional on any authenticated principal, creates Kerberoasting.
- RC4 encryption type support makes offline ticket cracking fast.
- Unconstrained delegation turns every authentication event into a potential TGT export.
- The PAC carries the authorization data that makes Windows access control work, and its integrity depends on the krbtgt key whose compromise enables Golden Tickets across the entire domain.

The next chapter builds on this protocol foundation to examine the enterprise identity services that sit above the protocols - the Public Key Infrastructure, Active Directory Certificate Services, Active Directory Federation Services, Microsoft Entra ID, hybrid identity, and the federation fabric - showing how the same trust mechanics that govern Kerberos extend into certificate-based authentication and cross-boundary federation, and where those extensions create the attack surface examined in detail in Part II.

ACRONYMS

AAL - Authenticator Assurance Level

AD - Active Directory

AES - Advanced Encryption Standard

AP-REQ - Application Request

AP-REP - Application Reply

AS - Authentication Service

AS-REQ - Authentication Service Request

AS-REP - Authentication Service Reply

DES - Data Encryption Standard

EPA - Extended Protection for Authentication

FAST - Flexible Authentication Secure Tunneling

HMAC - Hash-based Message Authentication Code

KDC - Key Distribution Center

LSASS - Local Security Authority Subsystem Service

MD4 - Message Digest 4

MD5 - Message Digest 5

NTLM - New Technology LAN Manager

PAC - Privilege Attribute Certificate

PBKDF2 - Password-Based Key Derivation Function 2

RBCD - Resource-Based Constrained Delegation

RC4 - Rivest Cipher 4

RFC - Request for Comment

SAM - Security Account Manager

SID - Security Identifier

SMB - Server Message Block

SPN - Service Principal Name

TGS - Ticket Granting Service

TGS-REQ - Ticket Granting Service Request

TGS-REP - Ticket Granting Service Reply

TGT - Ticket Granting Ticket

UPN - User Principal Name

REFERENCES

Neuman, B., Yu, T., Hartman, S., Raeburn, K. The Kerberos Network Authentication Service (V5). RFC 4120. July 2005. https://www.rfc-editor.org/rfc/rfc4120 (accessed July 14, 2026).

Microsoft. NTLM Overview. Microsoft Learn. https://learn.microsoft.com/windows-server/security/kerberos/ntlm-overview (accessed July 14, 2026).

Microsoft. Kerberos Authentication Overview. Microsoft Learn. https://learn.microsoft.com/windows-server/security/kerberos/kerberos-authentication-overview (accessed July 14, 2026).

Microsoft. Protected Users Security Group. Microsoft Learn. https://learn.microsoft.com/windows-server/security/credentials-protection-and-management/protected-users-security-group (accessed July 14, 2026).

Microsoft. How to configure Kerberos Constrained Delegation for Web Enrollment. Microsoft Learn. (accessed July 14, 2026).

Microsoft. Understanding Kerberos Double Hop. Microsoft Support. https://learn.microsoft.com/powershell/scripting/learn/remoting/ps-remoting-second-hop (accessed July 14, 2026).

Microsoft. MS14-068: Vulnerability in Kerberos Could Allow Elevation of Privilege. Microsoft Security Bulletin. https://docs.microsoft.com/security-updates/securitybulletins/2014/ms14-068 (accessed July 14, 2026).

National Security Agency. Detecting Abuse of Authentication Mechanisms. https://www.nsa.gov/ (accessed July 14, 2026).

RFC 6113 — A Generalized Framework for Kerberos Pre-Authentication (FAST). https://www.rfc-editor.org/rfc/rfc6113 (accessed July 14, 2026).

National Institute of Standards and Technology. Digital Identity Guidelines (SP 800-63B-4). July 2025. https://csrc.nist.gov/pubs/sp/800/63/b/4/final (accessed July 14, 2026).

What is a salt/nullbytes/concatenation/hash